

Trabajo de UML

Nombre

Sergio Andrés López Sepúlveda

Instructor

Luis Fernando Sánchez Pérez

Ficha

3169892

Servicio Nacional de Aprendizaje SENA

Centro De Tecnología De La Manufactura Avanzada CTMA

Análisis y Desarrollo De Software

2025

Tabla de Contenido

Introducción	4
Preguntas de Introducción:.....	5
¿Qué es un diagrama?	5
¿Qué tipos de diagramas conoce o ha utilizado?	5
¿Cómo se estructura un diagrama?	5
¿Cómo se construye un diagrama?	5
¿Cuál es el objetivo de usar diagramas?	6
¿Por qué consideras que deben usarse diagramas en el diseño de soluciones de software?	6
¿Cuáles son los tipos de diagrama más relevantes en UML? – Describa al menos 4 de ellos. ...	6
¿Cómo representar instancias de las clases en UML? - Elabore un ejemplo de una clase. Con sus atributos y características	7
¿UML permite incluir notas, cómo, para qué?	8
¿Qué es un método constructor y para qué se utiliza?	8
¿Qué tipos de Relaciones pueden especificarse entre clases?	8
Actividad 1: Diagrama de Casos de Uso en UML.....	9
¿Cómo se crea un Diagrama de Casos de Uso en UML y cuáles son los elementos que lo componen? Realice un diagrama para describir un proceso que identifique en su contexto (por ejemplo: el proceso para ir de compras al supermercado, un proceso de matrícula, el proceso para hacer un saque en un partido de tenis, etc.)	9
Elementos que componen un diagrama de casos de uso:	10
Relaciones:	10
Actividad 2: Diagrama de Clases en UML	11

Consulte cuales son los elementos que hacen parte de un diagrama de clases de UML y a partir de la actividad 1, construya una tabla en la cual identifique: verbos, sustantivos, y entidades que intervienen en el caso de uso descrito; con esa información construya el diagrama de clases correspondiente al proceso descrito.	11
Conclusiones	21
GLOSARIO	22
Referencias Bibliográficas	25

Introducción

Los diagramas son bastantes útiles para la creación y representación de procesos de forma simple, permiten poder demostrar el funcionamiento de alguna situación o describir cosas sobre algún tema, pero de manera gráfica y fácil de entender. Sin estos sería mucho más difícil de poder separar las partes de un tema y que sea fácil de seguir para otras personas ya que se tendría que explicar en una cantidad de tiempo mayor algo que pudo haber sido hecho de una manera más simple. En el desarrollo de cada pregunta se logrará comprender lo que es un diagrama, el cómo se hacen y para que usar estos.

Preguntas de Introducción:

¿Qué es un diagrama?

Un diagrama es una representación gráfica que permite visualizar información, procesos, estructuras o relaciones de manera simplificada. Se utiliza para organizar ideas, mostrar conexiones entre elementos o facilitar la comprensión de conceptos complejos. Los diagramas pueden ser jerárquicos, de flujo, de red, de Venn, entre otros, y son ampliamente usados en educación, ingeniería, informática y otras disciplinas.

¿Qué tipos de diagramas conoce o ha utilizado?

Conozco los diagramas de flujo, diagramas de Venn, el mapa conceptual o el diagrama conceptual, el mapa mental o diagrama radial y el diagrama circular.

¿Cómo se estructura un diagrama?

Un diagrama se estructura mediante elementos gráficos organizados que permiten representar información visualmente. Generalmente, incluye:

- Elementos o símbolos que representan conceptos, pasos o entidades.
- Conectores (como flechas o líneas) que muestran relaciones o secuencia.
- Texto explicativo para identificar o describir cada parte.
- Un diseño lógico y claro, que facilite la comprensión del contenido.

La estructura específica depende del tipo de diagrama, pero todos buscan comunicar información de forma visual y ordenada.

¿Cómo se construye un diagrama?

Construir un diagrama implica seguir una serie de pasos organizados para representar visualmente un concepto, proceso o sistema. Generalmente, el proceso incluye:

1. Definir el objetivo: Determinar qué se desea representar y para qué público.

2. Reunir la información: Identificar los elementos clave, relaciones y secuencias necesarias.
3. Elegir el tipo de diagrama adecuado: Por ejemplo, de flujo para procesos, de Venn para comparaciones, o entidad-relación para bases de datos.
4. Diseñar la estructura: Colocar los elementos en el orden lógico, usando símbolos y conectores apropiados.
5. Añadir etiquetas o texto: Incluir descripciones breves que clarifiquen los componentes.
6. Revisar y ajustar: Verificar que el diagrama sea claro, completo y fácil de entender.

¿Cuál es el objetivo de usar diagramas?

El objetivo principal de usar diagramas es facilitar la comprensión y comunicación de información compleja de manera visual. Los diagramas permiten simplificar conceptos, organizar información de manera clara y mejorar la visualización de relaciones y estructuras.

¿Por qué consideras que deben usarse diagramas en el diseño de soluciones de software?

Para tener una base con la que trabajar y una forma organizada de ver las ideas requeridas con las que uno puede encontrar una forma eficiente de resolver algún problema.

¿Cuáles son los tipos de diagrama más relevantes en UML? – Describa al menos 4 de ellos.

En UML (Unified Modeling Language), existen varios tipos de diagramas que se utilizan para representar diferentes aspectos de un sistema. Cuatro de estos son:

- Diagrama de clases: Representa las clases de un sistema, sus atributos, métodos y las relaciones entre ellas. Es uno de los diagramas más utilizados en la programación orientada a objetos.

- Diagrama de casos de uso: Muestra cómo los usuarios (actores) interactúan con el sistema. Este diagrama ayuda a entender los requisitos del sistema desde la perspectiva del usuario.
- Diagrama de secuencia: Representa la interacción entre objetos a lo largo del tiempo. Muestra cómo los mensajes se envían entre objetos y en qué orden, lo que es útil para modelar la lógica de interacción de un sistema.
- Diagrama de actividades: Representa el flujo de trabajo de un sistema o proceso. Es útil para modelar procesos de negocio o flujos de control dentro del sistema, mostrando actividades y decisiones.

¿Cómo representar instancias de las clases en UML? - Elabore un ejemplo de una clase.

Con sus atributos y características.

Para representar una clase se utiliza una caja que es dividida en tres zonas utilizando para ello líneas horizontales:

- La primera de ellas se utiliza para el nombre de la clase. En caso de que la clase sea abstracta se utilizará su nombre en cursiva.
- La segunda, por otra parte, se utiliza para escribir los atributos de la clase, uno por línea y utilizando el siguiente formato:

Visibilidad nombre_atributo : tipo = valor-inicial { propiedades }

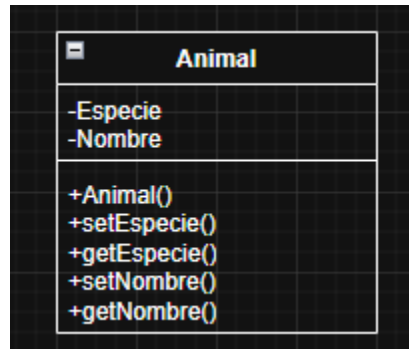
Es común simplificar poniendo solo el nombre y el tipo o únicamente el nombre.

- La última de las zonas incluye cada una de las funciones que ofrece la clase. sigue el siguiente formato:

Visibilidad nombre_funcion { parametros } : tipo-devuelto { propiedades }

Se suele simplificar indicando únicamente el nombre de la función y, en ocasiones, el tipo devuelto.

Ejemplo:



1 - draw.io ejemplo de una clase

¿UML permite incluir notas, cómo, para qué?

Si, UML permite incluir notas como elementos gráficos adicionales que se representan mediante un rectángulo con la esquina superior derecha doblada. Estas notas se utilizan para agregar comentarios, aclaraciones, restricciones o cualquier tipo de información adicional que no forma parte estructural del modelo, pero que puede ser útil para los diseñadores, programadores o lectores del diagrama.

¿Qué es un método constructor y para qué se utiliza?

Un método constructor es una función especial dentro de una clase que se utiliza para inicializar objetos cuando se crea una instancia de esa clase. Su función principal es asignar valores iniciales a los atributos del objeto y preparar su estado para ser usado en el programa.

¿Qué tipos de Relaciones pueden especificarse entre clases?

En UML, hay varios tipos de relaciones entre clases que indican cómo interactúan o se conectan dentro de un sistema. Las principales son:

- Asociación: Es la relación estructural más básica. Representa una conexión entre dos clases, como por ejemplo que un Profesor está asociado a varios Estudiantes. Puede tener multiplicidad (uno a uno, uno a muchos, etc.) y roles.
- Agregación: Es un tipo de asociación que representa una relación “todo/parte”, pero con independencia entre los objetos. Por ejemplo, una Universidad puede tener varios Departamentos, pero los departamentos pueden existir sin la universidad.
- Composición: Similar a la agregación, pero con una relación más fuerte. Indica que las partes no pueden existir sin el todo. Por ejemplo, un Libro compuesto por Capítulos: si se destruye el libro, también desaparecen sus capítulos.
- Generalización (o herencia): Representa una jerarquía entre clases, donde una clase hija hereda atributos y métodos de una clase padre. Por ejemplo, Empleado puede ser una clase base de Profesor y Administrativo.
- Dependencia: Muestra que una clase usa o depende temporalmente de otra para realizar una tarea, pero no forma parte permanente de su estructura.

Actividad 1: Diagrama de Casos de Uso en UML

¿Cómo se crea un Diagrama de Casos de Uso en UML y cuáles son los elementos que lo componen? Realice un diagrama para describir un proceso que identifique en su contexto (por ejemplo: el proceso para ir de compras al supermercado, un proceso de matrícula, el proceso para hacer un saque en un partido de tenis, etc.)

Un diagrama de casos de uso en UML se utiliza para representar las interacciones entre los actores (usuarios u otros sistemas) y el sistema, enfocándose en los requisitos funcionales. Para crearlo, se siguen los siguientes pasos:

- Identificar los actores: Son las entidades externas que interactúan con el sistema.

Pueden ser personas, sistemas u organizaciones.

- Identificar los casos de uso: Representan las funcionalidades o servicios que el sistema debe proporcionar a los actores.

- Establecer relaciones: Se dibujan las relaciones entre actores y casos de uso, y entre casos de uso entre sí, si corresponde.

- Dibujar el sistema: Se representa el sistema como una caja o contenedor que incluye todos los casos de uso.

Elementos que componen un diagrama de casos de uso:

- Actores: Representados con un ícono de persona o simplemente por su nombre.

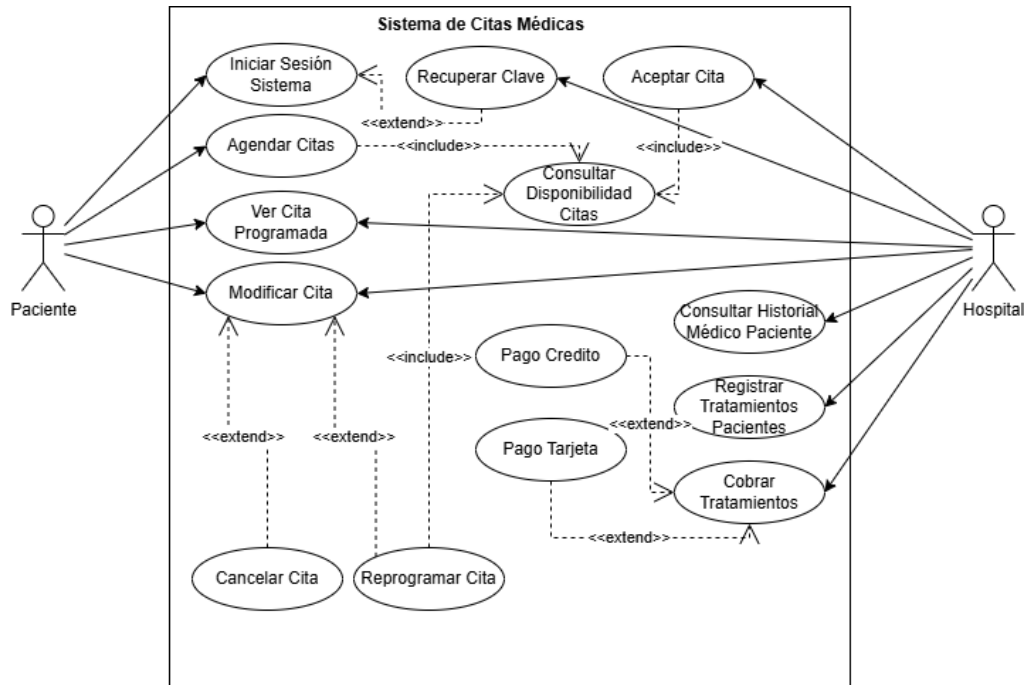
Son los que inician la interacción con el sistema.

- Casos de uso: Representados como óvalos con el nombre de la funcionalidad en su interior.

- Sistema: Representado como un rectángulo que contiene los casos de uso.

Relaciones:

- Asociación (línea simple): conexión entre un actor y un caso de uso.
- Inclusión (<<include>>): un caso de uso incluye obligatoriamente a otro.
- Extensión (<<extend>>): un caso de uso puede extenderse opcionalmente con otro.
- Generalización: un actor o un caso de uso hereda comportamientos de otro.



2 - draw.io ejemplo de un diagrama de caso de uso

Actividad 2: Diagrama de Clases en UML

Consulte cuales son los elementos que hacen parte de un diagrama de clases de UML y a partir de la actividad 1, construya una tabla en la cual identifique: verbos, sustantivos, y entidades que intervienen en el caso de uso descrito; con esa información construya el diagrama de clases correspondiente al proceso descrito.

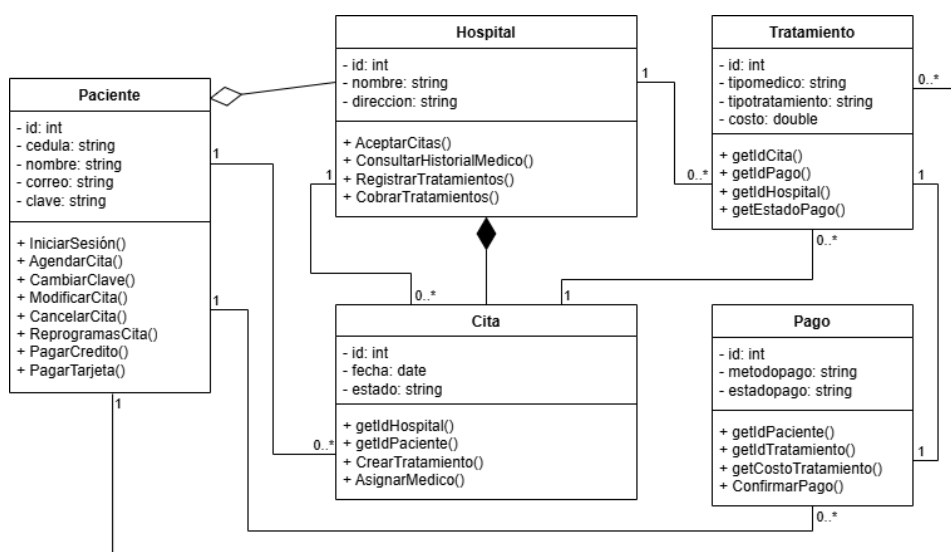
Un diagrama de clases en UML representa la estructura estática de un sistema, mostrando las clases, sus atributos, métodos y las relaciones entre ellas. Los principales elementos que lo componen son:

1. Clases: Son los bloques principales del diagrama. Cada clase se representa como un rectángulo dividido en tres secciones:
 - Nombre de la clase.
 - Atributos (propiedades o variables).
 - Métodos (operaciones o funciones).

2. Atributos: Son las características o datos que posee una clase. Se indican en la segunda sección del rectángulo, con su nombre y tipo (por ejemplo: nombre: String).
3. Métodos: Son las funciones que puede realizar la clase. Se escriben en la tercera sección del rectángulo, con su nombre, parámetros y tipo de retorno (por ejemplo: calcularEdad(): int).
4. Relaciones entre clases: Son conexiones que indican cómo se vinculan las clases entre sí. Las principales son:
 - Asociación: relación directa entre clases.
 - Agregación: relación “todo/parte” con independencia.
 - Composición: relación “todo/parte” con dependencia.
 - Generalización: herencia entre clases.
 - Dependencia: una clase usa a otra de forma puntual.
5. Multiplicidad: Especifica cuántas instancias de una clase pueden estar relacionadas con otra.
6. Visibilidad: Se indica con signos:
 - + para público
 - - para privado
 - # para protegido

Elemento	Verbo (acción)	Sustantivo (Objeto)	Entidad
Iniciar Sesión Sistema	Iniciar Sesión	Sistema	Paciente
Recuperar Clave	Recuperar	Clave	Paciente/Hospital
Agendar Citas	Agendar	Citas	Paciente

Consultar Disponibilidad Citas	Consultar	Disponibilidad de citas	Paciente/Hospital
Ver Cita Programada	Ver	Citas programadas	Paciente/Hospital
Modificar Cita	Modificar	Cita	Paciente/Hospital
Cancelar Cita	Cancelar	Cita	Paciente/Hospital
Reprogramar Cita	Reprogramar	Cita	Paciente/Hospital
Aceptar Cita	Aceptar	Cita	Hospital
Consultar Historial Médico Paciente	Consultar	Historial médico del paciente	Hospital
Registrar Tratamientos Pacientes	Registrar	Tratamientos del paciente	Hospital
Cobrar Tratamientos	Cobrar	Tratamientos	Hospital
Pago Crédito	Pagar	Crédito	Paciente
Pago Tarjeta	Pagar	Tarjeta	Paciente



3 - draw.io ejemplo de un diagrama de clases

Evidencias de la base de datos SQL y NoSQL

Base de datos SQL:

```

1 CREATE DATABASE MyHospital;
2 USE MyHospital;
3
4 CREATE TABLE hospital (
5     ID_HOSPITAL int(11) NOT NULL AUTO_INCREMENT,
6     NOMBRE varchar(100) NOT NULL,
7     DIRECCION varchar(255) NOT NULL,
8     PRIMARY KEY (ID_HOSPITAL)
9 );
10
11 CREATE TABLE paciente (
12     ID_PACIENTE int(11) NOT NULL AUTO_INCREMENT,
13     CEDULA varchar(100) NOT NULL,
14     NOMBRE varchar(100) NOT NULL,
15     CORREO varchar(100) NOT NULL,
16     CLAVE varchar(255) NOT NULL,
17     PRIMARY KEY (ID_PACIENTE),
18     UNIQUE KEY U_CEDULA (CEDULA),
19     UNIQUE KEY U_CORREO (CORREO)
20 );
21
22 CREATE TABLE cita (
23     ID_CITA int(11) NOT NULL AUTO_INCREMENT,
24     FECHA date NOT NULL,
25     ESTADO varchar(50) NOT NULL DEFAULT 'Pendiente',
26     ID_PACIENTE int(11) NOT NULL,
27     ID_HOSPITAL int(11) NOT NULL,
28     PRIMARY KEY (ID_CITA),
29     KEY FK_CITA_PACIENTEID (ID_PACIENTE),
30     KEY FK_CITA_HOSPITALID (ID_HOSPITAL)
31 );

```

Imagen 1. XAMPP - Creación base de datos

```

33 CREATE TABLE tratamiento (
34     ID_TRATAMIENTO int(11) NOT NULL AUTO_INCREMENT,
35     TIPO_MEDICO varchar(100) NOT NULL,
36     TIPO_TRATAMIENTO varchar(255) NOT NULL,
37     COSTO decimal(10,2) NOT NULL DEFAULT 0.00,
38     ID_CITA int(11) NOT NULL,
39     ID_HOSPITAL int(11) NOT NULL,
40     PRIMARY KEY (ID_TRATAMIENTO),
41     KEY FKTRATAMIENTO_CITAID (ID_CITA),
42     KEY FKTRATAMIENTO_HOSPITALID (ID_HOSPITAL)
43 );
44
45 CREATE TABLE pago (
46     ID_PAGO int(11) NOT NULL AUTO_INCREMENT,
47     METODO_PAGO varchar(50) NOT NULL DEFAULT 'Credito',
48     ESTADO_PAGO varchar(50) NOT NULL DEFAULT 'Por pagar',
49     ID_PACIENTE int(11) NOT NULL,
50     ID_TRATAMIENTO int(11) NOT NULL,
51     PRIMARY KEY (ID_PAGO),
52     KEY FK_PAGO_PACIENTEID (ID_PACIENTE),
53     KEY FK_PAGO_TRATAMIENTOID (ID_TRATAMIENTO)
54 );

```

Imagen 2. XAMPP - Creación ultimas tablas

```

56 ALTER TABLE cita
57   ADD CONSTRAINT FKcita_pacienteID FOREIGN KEY (ID_PACIENTE) REFERENCES paciente (ID_PACIENTE),
58   ADD CONSTRAINT FKcita_hospitalID FOREIGN KEY (ID_HOSPITAL) REFERENCES hospital (ID_HOSPITAL);
59
60 ALTER TABLE tratamiento
61   ADD CONSTRAINT FKtratamiento_citaID FOREIGN KEY (ID_CITA) REFERENCES cita (ID_CITA),
62   ADD CONSTRAINT FKtratamiento_hospitalID FOREIGN KEY (ID_HOSPITAL) REFERENCES hospital (ID_HOSPITAL);
63
64 ALTER TABLE pago
65   ADD CONSTRAINT FKpago_pacienteID FOREIGN KEY (ID_PACIENTE) REFERENCES paciente (ID_PACIENTE),
66   ADD CONSTRAINT FKpago_tratamientoID FOREIGN KEY (ID_TRATAMIENTO) REFERENCES tratamiento (ID_TRATAMIENTO);

```

Imagen 3. XAMPP - Creación llaves foráneas

```

1  INSERT INTO hospital (NOMBRE, DIRECCION) VALUES
2  ('Hospital Central', 'Calle 123 #45-67'),
3  ('Clínica Norte', 'Carrera 10 #20-30');
4
5  INSERT INTO paciente (CEDULA, NOMBRE, CORREO, CLAVE) VALUES
6  ('1234567890', 'Juan Pérez', 'juan@myemail.com', 'claveuno'),
7  ('9876543210', 'Ana López', 'ana@myemail.com', 'clavedos');
8
9  INSERT INTO cita (FECHA, ESTADO, ID_PACIENTE, ID_HOSPITAL) VALUES
10 ('2025-07-15', 'Pendiente', 1, 1),
11 ('2025-07-20', 'Confirmada', 2, 2);
12
13 INSERT INTO tratamiento (TIPO_MEDICO, TIPO_TRATAMIENTO, COSTO, ID_CITA, ID_HOSPITAL) VALUES
14 ('General', 'Consulta médica general', 50000.00, 1, 1),
15 ('Odontología', 'Limpieza dental', 75000.00, 2, 2);
16
17 INSERT INTO pago (METODO_PAGO, ESTADO_PAGO, ID_PACIENTE, ID_TRATAMIENTO) VALUES
18 ('Efectivo', 'Pagado', 1, 1),
19 ('Tarjeta', 'Por pagar', 2, 2);

```

Imagen 4. XAMPP - Agregación de datos

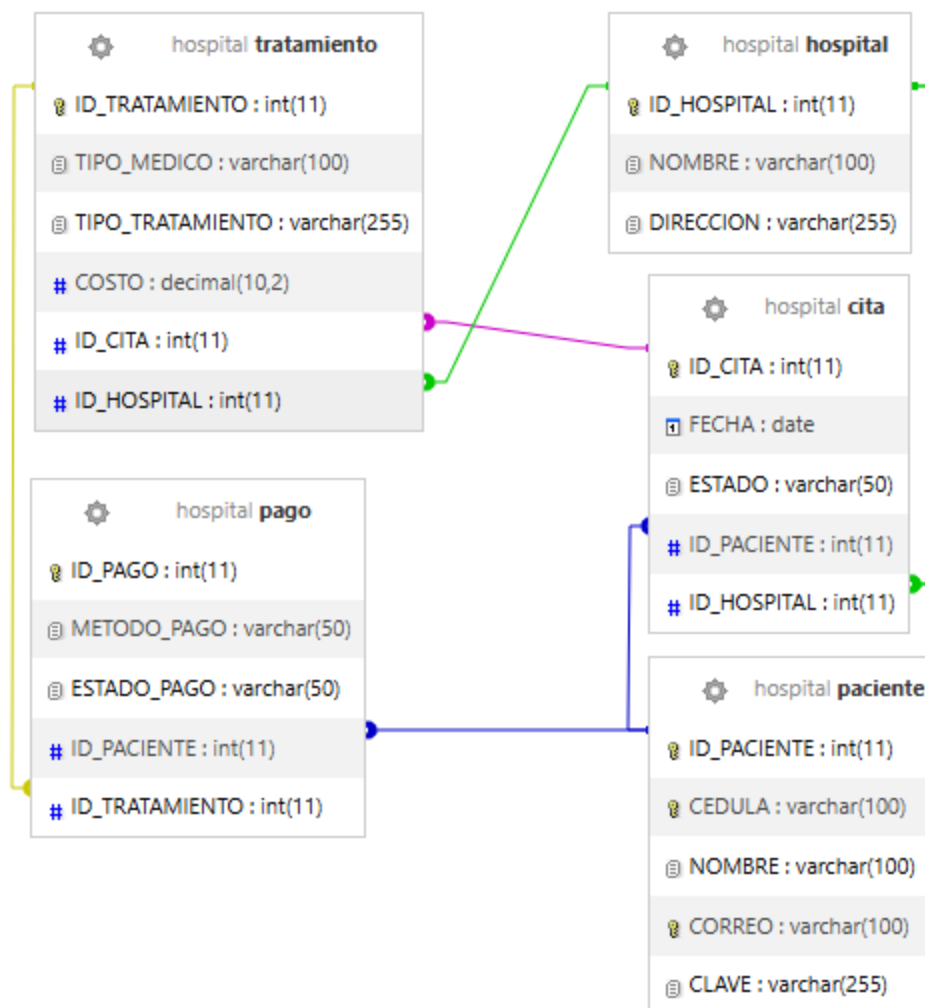


Imagen 5. XAMPP - Tablas graficas

```

1  SELECT
2    cita.ID_CITA,
3    cita.FECHA,
4    cita.ESTADO,
5    paciente.NOMBRE,
6    paciente.CEDULA,
7    hospital.NOMBRE,
8    hospital.DIRECCION
9  FROM cita
10 JOIN paciente ON cita.ID_PACIENTE = paciente.ID_PACIENTE
11 JOIN hospital ON cita.ID_HOSPITAL = hospital.ID_HOSPITAL;

```

Imagen 6. XAMPP – Ejemplo de consulta en SQL

ID_CITA	FECHA	ESTADO	NOMBRE	CEDULA	NOMBRE	DIRECCION
1	2025-07-15	Pendiente	Juan Pérez	1234567890	Hospital Central	Calle 123 #45-67
2	2025-07-20	Confirmada	Ana López	9876543210	Clínica Norte	Carrera 10 #20-30

Imagen 7. XAMPP - Tabla de la consulta SQL

```

1 SELECT
2   tratamiento.ID_TRATAMIENTO,
3   tratamiento.TIPO_MEDICO,
4   tratamiento.TIPO_TRATAMIENTO,
5   tratamiento.COSTO,
6   cita.FECHA,
7   paciente.NOMBRE,
8   hospital.NOMBRE AS HOSPITAL
9 FROM tratamiento
10 JOIN cita ON tratamiento.ID_CITA = cita.ID_CITA
11 JOIN paciente ON cita.ID_PACIENTE = paciente.ID_PACIENTE
12 JOIN hospital ON tratamiento.ID_HOSPITAL = hospital.ID_HOSPITAL;

```

Imagen 8. XAMPP - Ejemplo de otra consulta en SQL

ID_TRATAMIENTO	TIPO_MEDICO	TIPO_TRATAMIENTO	COSTO	FECHA	NOMBRE	HOSPITAL
1	General	Consulta médica general	50000.00	2025-07-15	Juan Pérez	Hospital Central
2	Odontología	Limpieza dental	75000.00	2025-07-20	Ana López	Clínica Norte

Imagen 9. XAMPP - Tabla de la otra consulta en SQL

Base de datos NoSQL:

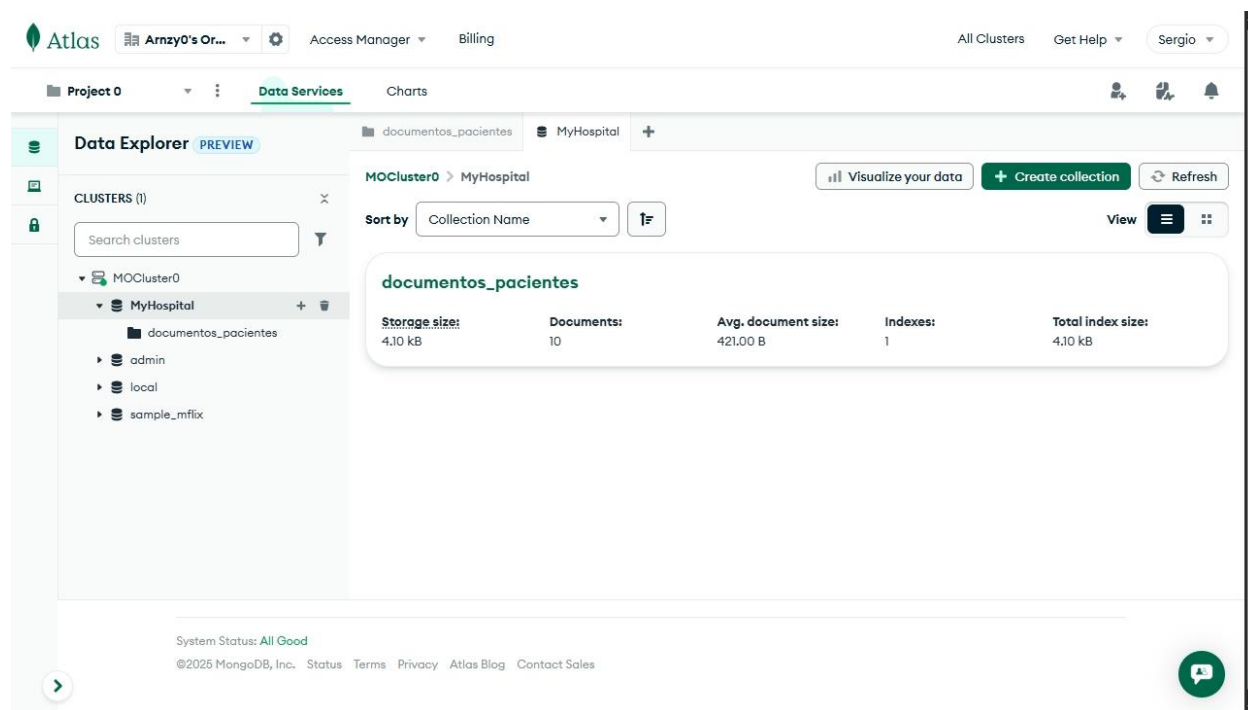


Imagen 10. MongoDB Atlas - Mi cluster con la base y la colección

```
_id: ObjectId('687050ae1b295c84f9a7d5cc')
cedula : "1234567890"
nombre : "Juan Pérez"
correo : "juan@myemail.com"
clave : "claveuno"
▸ hospital : Object
▸ citas : Array (1)
```

```
_id: ObjectId('687050ae1b295c84f9a7d5cd')
cedula : "9876543210"
nombre : "Ana López"
correo : "ana@myemail.com"
clave : "clavedos"
▸ hospital : Object
▸ citas : Array (1)
```

Imagen 11. MongoDB Atlas - Ejemplo de datos para una colección

```

_id: ObjectId('687050aeb295c84f9a7d5ce')
cedula : "3216549870"
nombre : "Carlos Ramírez"
correo : "carlosr@outemail.com"
clave : "clavetres"
▸ hospital : Object
▸ citas : Array (1)

```

```

_id: ObjectId('687050aeb295c84f9a7d5cf')
cedula : "4567891230"
nombre : "María González"
correo : "mariag@example.com"
clave : "clave321"
▸ hospital : Object
▸ citas : Array (1)

```

Imagen 12. MongoDB Atlas - Más ejemplos de datos en la colección

{"citas.estado": "Confirmada"}

+ ADD DATA ▾
UPDATE
DELETE
💡

_id: ObjectId('687050aeb295c84f9a7d5cc')
cedula : "1234567890"
nombre : "Juan Pérez"
correo : "juan@myemail.com"
clave : "claveuno"
▸ hospital : Object
▾ citas : Array (1)
 ▾ 0: Object
 fecha : "2025-07-15"
 estado : "Confirmada"
 ▸ tratamientos : Array (1)

_id: ObjectId('687050aeb295c84f9a7d5cf')
cedula : "4567891230"
nombre : "María González"
correo : "mariag@example.com"
clave : "clave321"
▸ hospital : Object
▾ citas : Array (1)
 ▾ 0: Object
 fecha : "2025-07-25"
 estado : "Confirmada"
 ▸ tratamientos : Array (1)

Imagen 13. MongoDB Atlas - Ejemplo de consulta de datos en la colección

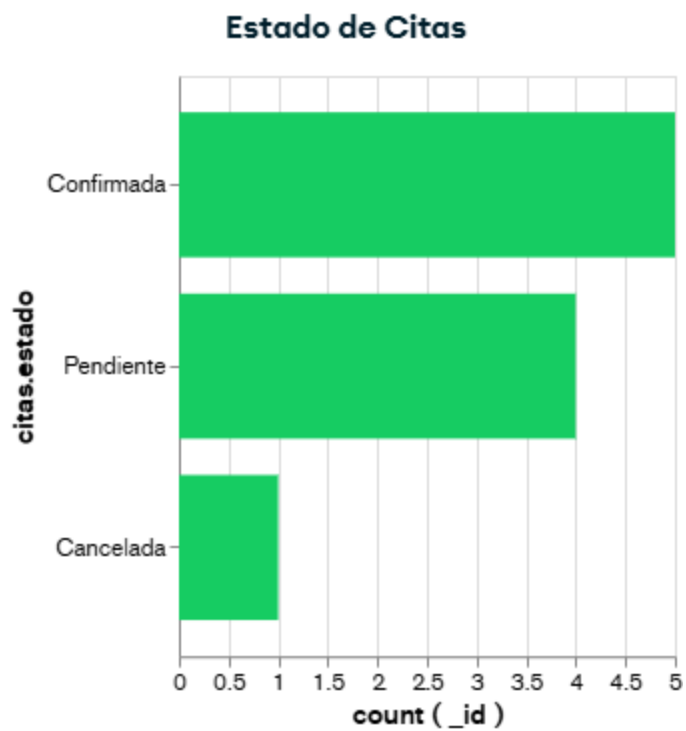


Imagen 14. MongoDB Atlas - Creación de una dashboard del estado de citas



Imagen 15. MongoDB Atlas - Ejemplo de otra dashboard del método de pago

Conclusiones

En conclusión, los diagramas son muy útiles para representar distintas situaciones hacia otras personas de manera comprensible y resumida, permite en distintos formatos lograrlo como los diagramas de tarta, de mapa conceptual, de mapa mental, de UML y muchos más. Usarlos según el contexto y usando las reglas correctas ayuda a que sean más eficientes que simplemente mostrar los datos en una hoja llena de ellos. Y gracias a todo eso es mucho más sencillo representar gráficamente distintas situaciones e informaciones.

GLOSARIO

- UML: el lenguaje unificado de modelado es un lenguaje gráfico para visualizar, especificar, construir y documentar un sistema. UML ofrece un estándar para describir un "plano" del sistema (modelo), incluyendo aspectos conceptuales tales como procesos, funciones del sistema, y aspectos concretos como expresiones de lenguajes de programación, esquemas de bases de datos y compuestos reciclados.
- POO: la programación orientada a objetos (POO) es un paradigma de programación que parte del concepto de "objetos" como base, los cuales contienen información en forma de campos (a veces también referidos como atributos, cualidades o propiedades) y código en forma de métodos.
- Objeto: en el paradigma de programación orientada a objetos (POO, o bien *OOP* en inglés), un objeto es un ente orientado a objetos (programa de computadoras) que consta de un estado y de un comportamiento, que a su vez constan respectivamente de datos almacenados y de tareas realizables durante el tiempo de ejecución.
- Clase: en informática, una clase es una plantilla para el objetivo de la creación de objetos de datos según un modelo predefinido. Las clases se utilizan para representar entidades o conceptos, como los sustantivos en el lenguaje. Cada clase es un modelo que define un conjunto de variables y métodos apropiados para operar con dichos datos.
- Modelo: un modelo es una forma de ver partes importantes de un sistema y no darles mucha importancia a otras. Lo creamos con herramientas de modelado que nos ayudan a hacer abstracciones de la realidad.

- **Atributo:** un atributo es una especificación que define una propiedad de un objeto, elemento o archivo. También puede referirse o establecer el valor específico para una instancia determinada de los mismos.

- **Alcance:** el alcance se refiere a la cantidad de datos que un dispositivo o sistema puede procesar o almacenar en un momento dado. Por ejemplo, un ordenador con un procesador rápido y una gran cantidad de memoria RAM tiene un mayor alcance que uno más antiguo y con menos recursos.

- **Parámetro:** un parámetro es un tipo de variable que es recibida por una función, procedimiento o subrutina.

Un parámetro influye en el comportamiento o el resultado de la ejecución de la función, procedimiento o subrutina (de ahora en más sólo procedimiento) que lo recibe. Son muy utilizados en la programación.

- **Agregación:** la agregación se refiere a la capacidad de combinar objetos o datos en una estructura más compleja. Por ejemplo, en las bases de datos relacionales, la agregación permite combinar y relacionar diferentes tablas para obtener resultados más significativos y completos.

- **Herencia:** la herencia es un mecanismo utilizado para la creación de objetos a partir de otros ya existentes e implica que una subclase obtiene todo el comportamiento (métodos) y finalmente los atributos (variables) de su superclase.

- **Polimorfismo:** se refiere a la propiedad por la que es posible enviar mensajes sintácticamente iguales a objetos de tipos distintos. Aunque el mensaje sea el mismo, diferentes objetos pueden responder a él de manera única y específica. Esta característica permite que, sin alterar ni tocar el código existente, se puedan incorporar nuevos

comportamientos y funciones (es decir la interfaz sintáctica se mantiene inalterada, pero cambia el comportamiento en función de qué objeto estamos usando en cada momento).

- Extensión: una extensión en informática es un archivo adicionado a un software o aplicación que contiene código adicional que se ejecuta en segundo plano, que modifica o agrega funcionalidad a la aplicación principal.

- Función: en programación, una función es un grupo de instrucciones con un objetivo en particular y que se ejecuta al ser llamada desde otra función o procedimiento. Una función puede llamarse múltiples veces e incluso llamarse a sí misma (función recursiva).

- Relación: En bases de datos, una relación es un vínculo entre entidades que describe una interacción entre ellas. Se pueden representar como tablas en SQL.

- Abstracción: La abstracción consiste en aislar un elemento de su contexto o del resto de los elementos que lo acompañan. En programación, el término se refiere al énfasis en el "¿qué hace?" más que en el "¿cómo lo hace?" (característica de caja negra).

- Sobrecarga: En programación orientada a objetos la sobrecarga se refiere a la posibilidad de tener dos o más funciones con el mismo nombre, pero funcionalidad diferente.

Es decir, dos o más funciones con el mismo nombre realizan acciones diferentes.

El compilador usará una u otra dependiendo de los parámetros usados. A esto se llama también sobrecarga de funciones.

Referencias Bibliográficas

- Arlow, J., & Neustadt, I. (2005). *UML 2 and the unified process: Practical object-oriented analysis and design* (2nd ed.). Addison-Wesley.
- Alegsa, L. (2023, 11 de junio). *Modelo (teoría de sistemas)*.
<https://www.alegsa.com.ar/Dic/modelo.php>
- Alegsa, L. (2023, 9 de julio). *Definición de parámetro (programación)*.
<https://www.alegsa.com.ar/Dic/parametro.php>
- Alegsa, L. (2023, 11 de junio). *Definición de función (programación)*.
<https://www.alegsa.com.ar/Dic/funcion.php#gsc.tab=0>
- Alegsa, L. (2023, 11 de octubre). *Significado de «agregación»*. Definiciones-de.com.
<https://www.definiciones-de.com/Definicion/de/agregacion.php>
- Alegsa, L. (2023, 9 de julio). *Definición de relación (base de datos relacional)*.
<https://www.alegsa.com.ar/Dic/relacion.php#gsc.tab=0>
- Booch, G., Rumbaugh, J., & Jacobson, I. (1999). *The unified modeling language user guide*. Addison-Wesley.
- Equipo editorial Etecé. (2022, 19 de julio). *Diagrama*. Enciclopedia Concepto.
<https://concepto.de/diagrama/>
- Fowler, M. (2004). *UML distilled: A brief guide to the standard object modeling language* (3rd ed.). Addison-Wesley.
- JGraph Ltd. (s.f.). *Draw.io*. <https://www.drawio.com/>
- Miñan, M. (2024). *Definición de alcance en informática: significado, ejemplos y autores*. DefinicionWiki.
<https://definicionwiki.com/definicion-de-alcance-en-informatica-significado-ejemplos-autores/>

Miñan, M. (2024). *Definición de extensión en informática: ejemplos, autores y concepto*.

DefinicionWiki.

<https://definicionwiki.com/definicion-de-extension-en-informatica-ejemplos-autores-concepto/>

Schildt, H. (2018). *Java: The complete reference* (11th ed.). McGraw-Hill Education.

Wikipedia. (2025, 8 de abril). *Lenguaje unificado de modelado*.

https://es.wikipedia.org/wiki/Lenguaje_unificado_de_modelado

Wikipedia. (2025, 15 de marzo). *Programación orientada a objetos*.

https://es.wikipedia.org/wiki/Programaci%C3%B3n_orientada_a_objetos

Wikipedia. (2025, 5 de febrero). *Objeto (programación)*.

[https://es.wikipedia.org/wiki/Objeto_\(programaci%C3%B3n\)](https://es.wikipedia.org/wiki/Objeto_(programaci%C3%B3n))

Wikipedia. (2023, 9 de diciembre). *Clase (informática)*.

[https://es.wikipedia.org/wiki/Clase_\(inform%C3%A1tica\)](https://es.wikipedia.org/wiki/Clase_(inform%C3%A1tica))

Wikipedia. (2023, 3 de febrero). *Atributo (informática)*.

[https://es.wikipedia.org/wiki/Atributo_\(inform%C3%A1tica\)](https://es.wikipedia.org/wiki/Atributo_(inform%C3%A1tica))

Wikipedia. (2024, 22 de noviembre). *Herencia (informática)*.

[https://es.wikipedia.org/wiki/Herencia_\(inform%C3%A1tica\)](https://es.wikipedia.org/wiki/Herencia_(inform%C3%A1tica))

Wikipedia. (2025, 25 de marzo). *Polimorfismo (informática)*.

[https://es.wikipedia.org/wiki/Polimorfismo_\(inform%C3%A1tica\)](https://es.wikipedia.org/wiki/Polimorfismo_(inform%C3%A1tica))

Wikipedia. (2025, 23 de febrero). *Abstracción (informática)*.

[https://es.wikipedia.org/wiki/Abstracci%C3%B3n_\(inform%C3%A1tica\)](https://es.wikipedia.org/wiki/Abstracci%C3%B3n_(inform%C3%A1tica))

Wikipedia. (2020, 2 de mayo). *Sobrecarga (informática)*.

[https://es.wikipedia.org/wiki/Sobrecarga_\(inform%C3%A1tica\)](https://es.wikipedia.org/wiki/Sobrecarga_(inform%C3%A1tica))